

Modifiche Sequence Diagrams

• Gestione Profilo Ristorante

1. `richiediProfilo()` → `ricercaRistorante()`
Per evitare ridondanza e ripetizione delle funzioni, la funzione `richiediProfilo` è stata sostituita con `ricercaRistorante`, già presente nel Domain Model, e che svolge la stessa operazione.
2. `getDati()`
dovrebbe richiederla direttamente il Ristore al Ristorante, il `GestoreRistoranti` ha solamente le responsabilità di creazione, ricerca ed eliminazione dei ristoranti, non di restituire i dati richiesti all'utente.
La freccia parte da Ristore direttamente verso Ristorante.
3. `salvaModifiche()` → `aggiornaDati()`
stesso discorso del punto 2, il `GestoreRistoranti` non aggiorna i dati del ristorante. La responsabilità di aggiornare i dati del Ristorante è del Ristorante stesso.
Ho modificato dunque il nome della funzione in `aggiornaDati ()`, più leggibile e generico.
4. `setDati()`
anche qui, superfluo il passaggio per il `GestoreRistoranti`. Inoltre, l'aggiornamento dei dati avviene già in `aggiornaDati ()`, dunque l'ho eliminata completamente.

NOTA: le responsabilità/funzioni che ritroviamo nel Domain Model presentano le parentesi alla fine (es. `funzione()`), mentre i messaggi che non sono “traducibili” in responsabilità/funzioni del Domain Model ma che riguardano più l'interfaccia grafica (es. `messaggioDiErrore`), oppure la restituzione di un oggetto o variabili (es. `Dati`), li ho rappresentati senza le parentesi `()` finali tipiche.

• Gestione Menu Inserimento Piatto

1. `apriGestioneMenu()`
Eliminato completamente, riguarda l'interfaccia grafica. Il `GestoreRistoranti` non si occupa di fornire all'utente l'interfaccia di gestione del menù.
`GestoreRistoranti` sarebbe dovuto essere presente solo nel momento in cui l'utente deve cercare il proprio ristorante da modificare, tuttavia è stato assunto nelle precondizioni che il Ristore si trovi già nell'interfaccia di gestione del menù. Dunque `GestoreRistoranti` non ha motivo di rimanere nel diagramma, e al posto suo va inserito direttamente il Ristorante.
2. `getMenu()`
Al posto di `GestoreRistoranti`, è il Ristorante che ha la responsabilità di restituire il proprio Menu. Se ce l'avesse `GestoreRistoranti`, dovrebbe per forza cercare il menù all'interno della propria lista di Ristoranti, il che è superfluo e complica il modello.
3. `getPiatti()`
Qui va bene, ma per i motivi prima elencati, deve partire direttamente da Ristore.
Ed è il Menu stesso a restituire i piatti, senza passare per `GestoreRistoranti`.
(Se proprio volessimo essere precisi, il caso d'uso riguarda l'inserimento di un piatto nel Menu, non la visualizzazione dei piatti, quindi sarebbe superfluo, però lo lascio lo stesso).

4. inserisciPiatto()

Anche qui la freccia va da Ristoratore a Menu.

NOTA: Teoricamente la freccia potrebbe andare anche da Ristoratore a Ristorante e poi da Ristorante a Menu, ma si complicherebbero le chiamate a funzioni che dovremmo in futuro implementare in Java. Esempio:

Se Ristoratore → Ristorante e Ristorante → Menu, si avrebbe un codice del tipo:

```
class Ristoratore {  
    Ristorante.getPiatti();  
}  
  
class Ristorante {  
    Menu.getPiatti();  
}
```

Se, invece si ha direttamente Ristoratore → Menu, si avrebbe un codice più leggibile del tipo:

```
class Ristoratore {  
    Ristorante.Menu.getPiatti();  
}
```

5. <<create>> creaPiatto()

Anche qui, la responsabilità di creare il piatto non è di GestoreRistoranti, a maggior ragione perché la classe Piatto è in composizione stretta con Menu, cioè non esiste al di fuori della classe Menu, e quindi deve essere creata, modificata e distrutta all'interno di Menu.

6. aggiungiPiatto()

Per il motivo spiegato nel punto 5, segue che aggiungiPiatto() è completamente superfluo visto che il piatto viene già creato all'interno di Menu.

7. confermaInserimento() e mostraMessaggioErrore()

Qui una semplice questione di leggibilità, come scritto nella nota, senza parentesi (), poiché non sono funzioni attribuibili a nessuna classe ma all'interfaccia.

- **Gestione Menu Modifica Piatto**

1. Fino a `mostraModuloPrecompilato()` tutto simile al sequence diagram precedente, quindi ho fatto le opportune modifiche.
Tuttavia, al posto di `getPiatti()`, conviene utilizzare direttamente `ricercaPiatto()`, per ottenere direttamente il piatto desiderato da modificare. Inutile ottenere tutta la lista, seppur a livello di interfaccia grafica può essere visualizzata l'intera lista, ciò rientra nell'implementazione, non nella progettazione ad alto livello.
2. `mostraModuloPrecompilato()`
Anche qui, riguarda l'implementazione di basso livello. Ad alto livello non serve sapere cosa visualizzerà l'utente finale sull'interfaccia, ma solo le interazioni tra le classi.
L'ho eliminata.
3. `salvaModifiche()` → `aggiornaDatiPiatto()`
In questo caso, la freccia va verso Menu, poiché il Piatto non esiste senza Menu (composizione stretta), dunque il Menu si occupa di fornire la funzionalità.
In `aggiornaDatiPiatto()` non può andare l'id del Piatto, poiché si tratta di un dettaglio implementativo che non può essere visibile. Per rimanere più generico possibile ho inserito semplicemente Piatto come parametro.

- **Gestione Menu Rimozione Piatto**

1. Fino a `mostraPopupConferma()` molto simile al precedente. L'ho modificato.
2. `mostraPopupConferma()`
Riguarda l'interfaccia grafica, rimosso.
3. `confermaEliminazione()`, `annullaEliminazione()`
Riguarda l'interfaccia grafica, nel sequence diagram di alto livello vanno specificate solo le interazioni significative tra le classi.
Ho dunque sostituito il frammento "alt" con due guardie differenti: "piatto esiste" e "piatto non esiste".

- **Accesso Utente**

1. `login(username, password)`
Considerando che `login()` non è una responsabilità di `GestoreUtenti`, conviene specificare tale azione come una semplice interazione tra Utente e sistema, senza richiamare la funzione `login()`. Dunque ho sostituito `login()` con "Inserisce credenziali", similmente a come già stato fatto per i casi d'uso di Registrazione Utente.
2. `identificaRuolo()`
Il `GestoreUtenti` non può identificare il ruolo in una chiamata su se stesso (la freccia verso se stesso), al massimo dovrebbe esserci una freccia da `GestoreUtenti` verso Utente, ma lo reputo superfluo. Si potrebbe semplificare questa funzione semplicemente inserendo il ruolo già nel login, immaginando che in una futura implementazione vi siano interfacce diverse per ogni ruolo oppure un semplice campo dell'interfaccia di login con le tre opzioni.
3. `mostraInterfaccia()`
Per gli stessi motivi spiegati prima, va rimosso poiché riguarda l'interfaccia grafica e non una responsabilità delle classi del Domain Model. L'ho sostituito con "AccessoConsentito".

- **Effettua Ordine**

1. `consultaMenu()` → `getMenu()`

Per evitare ripetizioni di responsabilità con la stessa funzione, sostituisco `consultaMenu()` con `getMenu()`, già precedentemente utilizzata e che ha lo stesso identico scopo.

Stessa cosa anche per `ottieniPiatti()` → `getPiatti()`.

2. `creaOrdine()` → `inviaOrdine()`

`GestoreOrdini` aveva due funzioni con lo stesso nome, una `creaOrdine()` attivata dall'utente Cliente per confermare l'ordine e una per creare un oggetto della classe `Ordine`.

La prima l'ho rinominata in `inviaOrdine()`.

- **Elimina Utente**

1. Ho solamente cambiato il nome delle responsabilità/funzioni per essere più generici e coerenti con il resto del progetto.

2. Ho aggiunto un controllo if per verificare che l'utente esista effettivamente prima di eliminarlo.

- **Gestione Ordini Ricevuti (Da rivedere)**

1. C'è inconsistenza tra Use Case Diagram e Sequence Diagram. Nello Use Case il Ristoratore è attore primario, mentre nel Sequence si vede che il caso d'uso è inizializzato da un Found Message.

Ho pensato di semplificare tutto rimuovendo il sistema di notifica che complica la situazione, ipotizzando semplicemente che il Ristoratore visualizzi l'elenco degli ordini in stato "Inviato", in modo da far partire lui il caso d'uso.

Il sistema di notifica non viene rimosso del tutto, ma solamente dal caso d'uso in questione, che quindi avrà come preconditione il fatto che il Ristoratore abbia già ricevuto la notifica.

2. `ricercaOrdine()`

Avendo modificato i passaggi precedenti, è il Ristoratore stesso ad avviare la funzione di `ricercaOrdine` all'interno di `GestoreOrdini`.

In questo modo diventano superflui `mostraElencoOrdini()` (relativo sempre all'interfaccia grafica) e `selezioneOrdine(id)`, che svolge la stessa funzione di `ricercaOrdine()` che parte da Ristoratore.

NOTA: In `selezioneOrdine(id)`, ad alto livello di progettazione del Domain Model, non può essere passato l'id dell'ordine come parametro in entrata. L'id riguarda l'implementazione di dettaglio che nel Domain Model non deve proprio apparire.

3. `accettaOrdine()` e `rifiutaOrdine()`

Come già spiegato in casi d'uso simili precedenti, non è l'information expert, in questo caso il `GestoreOrdini`, ad avviare `accettaOrdine()`, ma direttamente il Ristoratore.

4. `inviaNotificaConferma()` e `inviaNotificaRifiuto()`

Anche in questo caso, probabilmente sarebbe più facile inserire l'invio della notifica all'interno delle postcondizioni del caso d'uso, poiché nessuna classe del Domain Model dovrebbe avere tale responsabilità, che dovrebbe appartenere invece ad un sistema di notifiche esterno.

Tuttavia si può implementare un `messaggioDiConferma` o `messaggioDiRifiuto` solo per motivi di leggibilità, che non sono funzioni/responsabilità di nessuna classe.

5. aggiornaPassaggioStato()

Risulta scomodo avere questa funzione dopo l'if/else, poiché ci si dovrebbe preoccupare di passare come parametro della funzione il nuovo stato dell'ordine.

Sarebbe più semplice aggiornare lo stato dell'ordine direttamente tramite le funzioni accettaOrdine() e rifiutaOrdine(), per evitare ridondanza di funzioni simili.

6. Attore secondario Cliente

Inconsistenza tra Use Case Diagram e Sequence Diagram: Se il Cliente deve ricevere la notifica all'interno del Sequence Diagram dovrebbe risultare attore secondario del caso d'uso all'interno dello Use Case Diagram.

Dunque vi sono due soluzioni: aggiungere il Cliente come attore secondario, oppure rimuovere Cliente dal Sequence Diagram.

Ne possiamo discutere insieme.

- Monitoraggio Stato Ordine (Da rivedere)

1. richiestaMonitoraggio(idOrdine)

Anche qui, non va bene passare idOrdine come parametro. Inoltre, considerando che subito dopo vi è un'altra richiesta recuperaDatiOrdine(), risultano superflue entrambe, dunque ho direttamente eliminato richiestaMonitoraggio() e l'ho sostituita con ricercaOrdine(), che mancava.

2. recuperaDatiOrdine() → getStatoOrdine()

Dovrebbe partire da Cliente, non da GestoreOrdini. Ho cambiato il nome in getStatoOrdine(), visto che negli altri casi abbiamo usato la keyword "get", per mantenere coerenza.

3. Presenza di Ristoratore

Penso che il Ristoratore non debba essere presente in questo caso d'uso: Il Monitoraggio Stato Ordine riguarda semplicemente la visualizzazione dello stato dell'ordine da parte del cliente.

Inserendo il Ristoratore che aggiorna lo stato, significa che ogni volta che il Cliente visualizza lo stato dell'ordine il Ristoratore ne aggiorna lo stato.

Andrebbe fatto un caso d'uso a parte per il Ristoratore per gestire l'aggiornamento dello stato dell'ordine.

Perciò ho eliminato tutte le frecce che riguardavano questo, e dunque anche il Ristoratore come attore secondario, e ho aggiunto un caso d'uso che permette al Ristoratore di aggiornare lo stato dell'ordine da "InPreparazione" a "InConsegna" (Avvia Consegna Ordine).

Ora però, sorge il dubbio su come far passare lo stato dell'ordine da "InConsegna" a "Consegnato": dovrebbe farlo il Ristoratore oppure è il Cliente che conferma di aver ricevuto l'ordine?

- Ricerca Ristoranti

1. ricercaRistorante() → getRistorantiAperti()

Sostituito con getRistorantiAperti(), più coerente con quello che fa effettivamente il caso d'uso.

2. recuperaListaRistoranti()

Superfluo, il GestoreRistoranti ha già la lista dei ristoranti perché ne è l'information expert.

3. verificaApertura() → isAperto()

Considerando che in una futura implementazione si tratterà di una funzione che ritorna un valore booleano True o False, ho cambiato il nome della funzione in un nome più leggibile. Inoltre, isAperto() va chiamata su tutti i ristoranti esistenti, quindi la chiamata a funzione andrebbe inserita in un ciclo for.

4. recuperaOrariApertura

La classe OrarioApertura è l'orario di apertura di un ristorante in un determinato giorno della settimana, non può avere la responsabilità di restituire tutti gli orari di apertura della settimana. In questo senso, è come se il Ristorante fosse information expert degli orari di apertura. Dunque, al massimo potrebbe essere una chiamata ricorsiva a se stesso, ma sarebbe inutile in quanto, essendo l'information expert degli orari di apertura, possiede già la lista degli orari di apertura, non deve "recuperarla".

In pratica, la classe OrarioApertura va eliminata dal Sequence Diagram poiché non vi è nessuna interazione tra l'oggetto Ristorante ed un oggetto OrarioApertura, ovvero una singola istanza di un orario di apertura.

5. Frammento if

Avendo inserito un ciclo, il frammento if che gestisce la risposta all'utente in base ai ristoranti trovati, conviene inserirlo all'interno del for.

In pratica, ho sostituito un controllo "finale" su tutta la lista dei ristoranti, con un controllo "singolare", all'interno del ciclo, che restituisce al GestoreRistoranti solamente i singoli ristoranti aperti, in modo che il GestoreRistoranti possa restituire al Cliente, dopo il ciclo, direttamente la lista dei ristoranti aperti.

Modifiche Domain Model

1. Associazione Ristorante-Menu

Modificata da composizione stretta a semplice associazione uno-a-uno per motivi di semplicità. Utilizzare la composizione avrebbe comportato una composizione (Menu-Piatto) nella composizione (Ristorante-Menu), e quindi la responsabilità di modificare un Piatto del Menu sarebbe spettata al Ristorante, risultando troppo “pieno” di responsabilità.

2. Modifiche generali a tutte le responsabilità e funzioni per essere coerenti con quelle presenti nei Sequence Diagram.

Modifiche Use Case Diagram

1. Monitoraggio Stato Ordine

Eliminazione di Ristoratore come attore secondario.

2. Aggiunta caso d'uso Avvia Consegna Ordine

Permette al Ristoratore di aggiornare lo stato dell'Ordine da “InPreparazione” a “InConsegna”.